

A. Data Solution Design

A credit company must submit daily loan performance data to multiple regulatory portals. Existing source data resides in the **raw** schema of the data warehouse. Currently, the process suffers from duplicate records, slow performance, inconsistent payment states, and manual submission. `ingestion_date` represents the warehouse ingestion/update date and is the partitioning column.

Design two parts:

- Suppose your reporting system is already built-up, and only require queries to pull the data. Design solutions and strategies to fulfill the regulatory reporting requirements below.
- If there are 10 other unique regulatory reporting pipeline, how would you handle the situation differently

To answer this case:

- State any assumptions you established
- Attach queries / pseudo-queries (not mandatory to be executable), and any supporting diagrams
- Describe the tools you would use to accomplish this (if any).

Source tables:

raw.loan_accounts

Grain: one record per loan state/version.

Size: 2 TB

Column	Type	Description
<code>loan_id</code>	INT	Loan ID
<code>user_id</code>	INT	User ID related to the loan
<code>product_id</code>	INT	Loan product
<code>status</code>	STRING	ACTIVE, OVERDUE, PAID.
<code>principal_amount</code>	FLOAT	Original loan principal
<code>disbursed_at</code>	TIMESTAMP	Loan disbursement timestamp
<code>ingestion_date</code>	DATE	Reepresents the warehouse ingestion/update date and is the partitioning column.

Example:

loan_id	user_id	product_id	status	principal_amount	disbursed_at	ingestion_date
L001	U001	P01	ACTIVE	10000000.0	2026-09-11 14:56:32	2026-09-12
L001	U001	P01	OVERDUE	10000000.0	2026-09-11 14:56:32	2026-09-12
L001	U001	P01	PAID	10000000.0	2026-09-11 14:56:32	2026-09-12

raw.users

Grain: one record per user. A user_id may have multiple versions as its data updates over time.

Size: 50GB

Column	Type	Description
user_id	INT	Unique user identifier
name	STRING	User name
date_of_birth	DATE	Date of birth
occupation	STRING	Occupation
risk_segment	STRING	Risk classification
ingestion_date	DATE	Represents the warehouse ingestion/update date and is the partitioning column.

raw.loan_payments

Grain: one record per payment transaction. A payment_id may have multiple versions as its status changes over time.

Size: 3TB

Column	Type	Description
payment_id	INT	Unique payment identifier
loan_id	INT	Loan identifier
payment_date	DATE	Payment date
amount	INT	Payment amount
payment_status	STRING	SUCCESS, FAILED, REVERSED
ingestion_date	DATE	Date this row is created or updated

Regulatory Output A: Daily Loan Performance

Grain: report_date × loan_id

Expected output:

Column	Description
report_date	Reporting date
loan_id	Loan identifier
user_id	User owning the loan
product_type	Loan product type
loan_status	Loan status as of report date
principal_amount	Original principal
total_paid	Successful payments up to report date
outstanding_principal	Principal minus total paid

Regulatory Output B: Daily User Loan Summary

Grain: report_date × user_id

Expected output:

Column	Description
report_date	Reporting date
user_id	User identifier
name	User name
risk_segment	User risk segment

active_loan_count	Number of active loans
overdue_loan_count	Number of overdue loans
total_principal	Total principal across loans
total_outstanding	Total outstanding balance

B. Reverse ETL at scale

Design a Python-based reverse-ETL system that moves data from a warehouse into multiple destinations.

System must support hundreds of pipelines. Pipelines may have different schedules, queries, mappings, and destinations.

Consider:

- Modularity and extensibility
- Incremental processing
- Configuration and secrets
- Reliability
- Further development efficiency

To answer this case:

- **Do not provide complete code**, instead answer in architect's POV, and provide a high-level view of your system (code snippets, file structure, architecture diagram, etc).
- Use Apache Airflow or equivalent tools.
- External services may be mocked.
- State any assumptions made